

A számításelmélet alapjai I.

12. előadás

előadó: Tichler Krisztián
ktichler@inf.elte.hu

Példa BNF-re

Egyszerűbb írásmód: alternatívák, kiemelések (csoport)

$\langle \text{azonosító} \rangle ::= \langle \text{betű} \rangle | \langle \text{betű} \rangle \langle \text{azonosítónév} \rangle$

$\langle \text{azonosító} \rangle ::= \langle \text{betű} \rangle \{ | \langle \text{azonosítónév} \rangle \}$

$\langle \text{azonosítónév} \rangle ::= \langle \text{betű} \rangle \langle \text{számjegy} \rangle | \langle \text{betű} \rangle \langle \text{azonosítónév} \rangle |$
 $\langle \text{számjegy} \rangle \langle \text{azonosítónév} \rangle$

$\langle \text{azonosítónév} \rangle ::= \{ \langle \text{betű} \rangle | \langle \text{számjegy} \rangle \} \{ | \langle \text{azonosítónév} \rangle \}$

$\langle \text{betű} \rangle ::= a | b | \dots | z$

$\langle \text{számjegy} \rangle ::= 0 | 1 | \dots | 9$

Az ismétlődéseket a BNF rekurzióval kezeli!

Pl. $\langle \text{azonosítónév} \rangle$

$\langle \text{azonosítónév} \rangle \rightarrow \langle \text{betű} \rangle \langle \text{azonosítónév} \rangle \rightarrow \langle \text{betű} \rangle \langle \text{betű} \rangle \langle \text{azonosítónév} \rangle \rightarrow$

$\rightarrow \langle \text{betű} \rangle \langle \text{betű} \rangle \langle \text{betű} \rangle \langle \text{azonosítónév} \rangle \rightarrow \dots$

BNF – Grammatikák a gyakorlatban

BNF (Backus-Naur Form) **John Backus, Peter Naur ALGOL 60**

A BNF egy széles körben használt metanyelv melynek segítségével szabályok alkothatók meg (például egy programozási nyelv szintaktikai szabályai). Építőkövei:

- ▶ $\langle \text{név} \rangle$, **fogalmak** (vagy más néven **nemterminálisok**)
- ▶ $::=$, a szabályok bal- és jobboldalának elválasztására
- ▶ a sztringek alkotóelemei (**terminálisok**)

Egy szabály bal- és jobboldalból áll, köztük $::=$, baloldalon pontosan 1 fogalom, jobboldalon terminálisok és fogalmak véges sorozata állhat.

A BNF megfelel a környezetfüggetlen grammatikának. Példa:

$\langle \text{azonosító} \rangle ::= \langle \text{betű} \rangle | \langle \text{betű} \rangle \langle \text{azvég} \rangle$

$\langle \text{azvég} \rangle ::= \langle \text{betű} \rangle | \langle \text{számjegy} \rangle | \langle \text{betű} \rangle \langle \text{azvég} \rangle | \langle \text{számjegy} \rangle \langle \text{azvég} \rangle$

$\langle \text{betű} \rangle ::= a | \dots | z$

$\langle \text{számjegy} \rangle ::= 0 | \dots | 9$

EBNF – A BNF kiterjesztése

EBNF: **Extended Backus-Naur Form**

iteratív változat

$\langle \text{azonosító} \rangle ::= \langle \text{betű} \rangle @ \{ \langle \text{betű} \rangle | \langle \text{számjegy} \rangle \}$

akárhány lépésű iteráció.

A „@” utáni terminálist, nemterminálist vagy csoportot akárhányszor ismételhetjük (a 0-t is beleértve).

$\langle \text{azonosító} \rangle ::= \langle \text{betű} \rangle \{ \langle \text{betű} \rangle | \langle \text{számjegy} \rangle \}_0^5$

legfeljebb 5, legalább 0 ismétlődés.

A terminálist, nemterminálist vagy csoportot legalább illetve legfeljebb annyiszor ismételhetjük mint az alsó illetve felső indexe.

Ami EBNF-fel leírható az BNF-fel is, de EBNF-fel esetleg egyszerűbben, tömörebben.

EBNF – tömörebb reprezentáció

1. Példa: Nemnegatív valós számok BNF-fel

```
<törtszám> ::= <egész>.|<egész>.<egész>|.<egész>  
<egész> ::= <számjegy>|<számjegy><egész>  
<számjegy> ::= 0|1|...|9  
<előjel> ::= +|-  
<valósszám> ::= <egész>E<előjel><számjegy>|  
                <egész>E<előjel><számjegy><számjegy>|  
                <törtszám>|  
                <törtszám>E<előjel><számjegy>|  
                <törtszám>E<előjel><számjegy><számjegy>
```

2. Példa: Nemnegatív valós számok EBNF-fel

```
<törtszám> ::= <egész>.{|<egész>|.}<egész>  
<egész> ::= <számjegy>@<számjegy>  
<számjegy> ::= 0|1|...|9  
<valósszám> ::= <törtszám>|  
{<törtszám>|<egész>}E{+|-}<számjegy>{|<számjegy>}
```

Szintaxis gráfok

Az EBNF-fel történő leírás szemléletesebb, gráfok segítségével történő megadása.

A gráf irányított, összefüggő, egyetlen forrással és nyelővel. Az élek címkézetlenek, az szögpontok vagy címkézetlenek, vagy □-be, vagy ○-be írt címkékkel ellátottak.

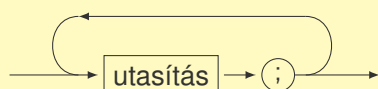
- ▶ A fogalmak □ címkéjű csomópontokba,
- ▶ a terminálisok (vagy terminális sorozatok) ○ címkéjű csomópontokba kerülnek,
- ▶ a gráf neve a definiálandó fogalom.

A gráf a következő értelemben írja le a definiálandó fogalmat: Tekintsünk egy irányított sétát a forrástól a nyelőig. Az séta mentén a címkézett csúcsoknak megfelelő fogalmak és terminálisok konkatenáltja meghatároz egy sorozatot (szót). A definiálandó fogalmat az összes ilyen séta által meghatározott sorozatok összessége írja le.

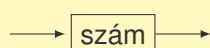
Szintaxisgráf – példa

Egy egyszerű programozási nyelv alapfogalmai:

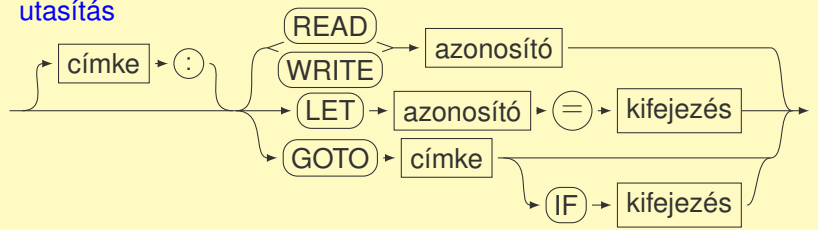
egyszerű program



címke



utasítás



BNF, EBNF, szintaxisgráf és a 2-típusú nyelvek

Megjegyzés: A BNF, az EBNF, a szintaxisgráfok és a környezetfüggetlen grammatikák azonos leíró erővel bírnak.

Megjegyzés: A környezetfüggetlen grammatikák azonban általában önmagukban nem elegendőek programozási nyelvek szintaxisának leírásához.

Tudjuk, hogy $\{ww \mid w \in V^*\} \notin \mathcal{L}_2$ ha $(|V| \geq 2)$. (A Nagy Bar-Hillel lemmával bizonyítható.)

Ez a nyelv a programozási nyelvek deklaráció fogalmának modellje, ahol az első w a deklarációt, a második w a használatot jelenti.

A fordítóprogramok működésének alapjai

A fordítóprogramok az inputként kapott karaktersorozatot az adott programozási nyelv szabályai alapján szintaktikusan ellenőrzik és a helyesnek bizonyuló programokból elkészítik a tárgykódot.

A fordítás lépései (általában):

1. A **lexikális elemzés** során a fordító felismeri a programnyelv lexikális egységeit (azonosítók, alapszavak, ...) és azokat táblázatban rögzítve a felsőbb szintek számára érthető kódjukkal helyettesíti. A lexikális analízis egy 3-as típusú (**reguláris**) nyelv szavainak felismerésén alapszik. Ez nyelvet felismerő determinisztikus véges automata programnyelvi eszközökkel történő szimulációjával viszonylag könnyen megvalósítható.
2. A **szintaktikus elemző** a lexikálisan elemzett, átkódolt szimbólumsorozatot szerkezetileg ellenőrzi a nyelvet leíró 2-es típusú grammatika felhasználásával. Ennek az elemzési fázisnak egy **levezetési fa** a kimenete.

A fordítóprogramok működésének alapjai

3. A **szemantikus elemző** szemantikusan ellenőrzi a felépített levezetési fát. Ebben a fázisban kerül sor az esetleges típusellenőrzésre, annak ellenőrzésére, hogy a változó- és függvényreferenciák megfelelnek-e a definíciójuknak, a lokális változók inicializálva vannak-e, stb.
4. **Kódoptimalizálás.**
5. A **tárgykód elkészítése** (assembly kód).

Az 1-3 lépések tehát ellenőrzik a programkód szintaktikai és szemantikai helyességét, ez a fordító **front end** része, a 4-es lépés a **middle end**, az 5-ös a **back end**.

Az 1-3 lépések bármelyike végződhet sikertelenül, ilyenkor az elemzést hibaüzenettel leállítjuk. Az 1. és 2. lépéseknél talált hiba esetén szintaktikusan, a 3. lépésnél talált hiba esetén szemantikusan hibás a bemenet.

Mi most a csak a 2-es lépés (szintaktikus elemzés) elméleti hátterét vizsgáljuk majd.

A szintaxiselemzési módszerek osztályozásai

Az elemzés során tehát az adott $G = (N, \Sigma, P, S)$ 2-es típusú grammatika és $u \in \Sigma^*$ szó esetén meg kell kísérelni felépíteni u egy levezetési fáját. Ha ez sikerült, az elemzés sikeres volt, egyébként pedig a szó szintaktikusan hibás.

A levezetési fa felépítése történhet

- ▶ felülről lefelé (**top-down**) elemzés révén: a nyelv főfogalmából vagyis a gyöktérből kiindulva építjük fel a fát,
- ▶ alulról-felfelé (**bottom-up**) elemzés révén: a fa leveleiből kiindulva építjük fel a fát.

Az input karaktersorozat elemeihez való hozzáférés lehet

- ▶ közvetlen
- ▶ csak valamelyik irányból szekvenciálisan olvasható (csak 1x)
- ▶ csak valamelyik irányból szekvenciálisan olvasható (többször is)

A szintaxiselemzési módszerek osztályozásai

Leggyakrabban az elemzendő program szövege szekvenciális input file-ban érhető el, s a nagy méret alapján nem gazdaságos a többszöri végigolvasás, ezért mi csak olyan elemzési módszerekkel foglalkozunk, ahol az input szót csak balról jobbra, egyszer lehet olvasni. Ezeket az elemzőket balról-jobbra (**left to right**) elemzőknek nevezzük.

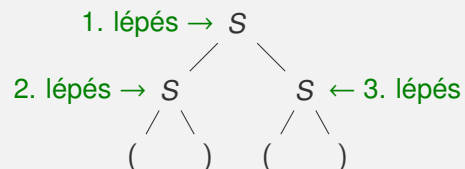
A továbbiakban a top-down elemzésekkel foglalkozunk részletesebben.

A levezetési fa fogalmát a korábban definiáltnál néha általánosabban értelmezzük: nem csak terminális szavaknak, hanem mondatformáknak is lehet levezetési fája, ilyenkor persze a levelek nemterminálisok is lehetnek.

Top-down elemzés példa

Először nézzük egy példát. Legyen $\Sigma = \{ (,) \}$, $N = \{ S \}$, $u = ()()$, és legyen $G_1 = (N, \Sigma, P, S)$, ahol $P = \{ S \rightarrow SS \mid (S) \mid () \}$. Ekkor a levezetési fa felépítése S kezdőszimbólumból az

$S \Rightarrow SS \Rightarrow ()S \Rightarrow ()()$ **baloldali** levezetést követve az alábbi:



A célunk tehát $(\mathcal{S}, \mathcal{S})$ egy levezetési fájának a megtalálása S -ből.

Észrevétel: Elegendő baloldali levezetést keresni, hiszen minden levezethető szónak van baloldali levezetése is.

Keresési stratégiák a lefutási fában

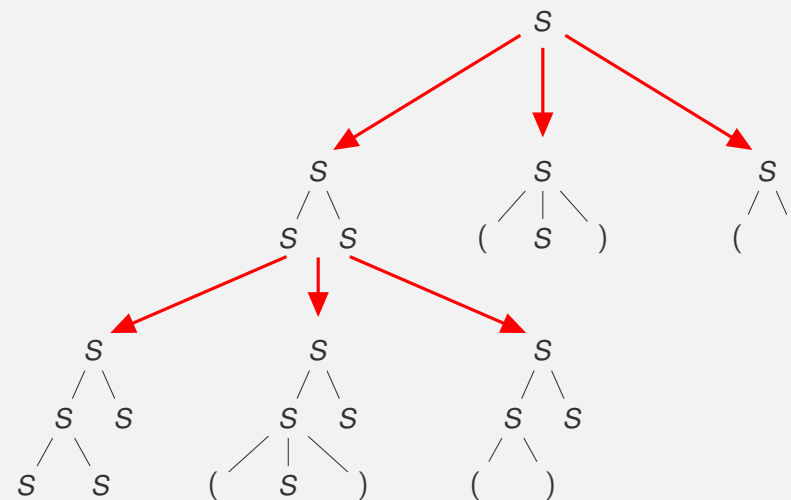
A lefutási fa gyakran végtelen. Olyan determinisztikus algoritmus kéne, amelyik a lefutási fa csúcsait valamilyen szisztematikus sorrend szerinti bejárja. Ha a bejárás során valamikor u egy levezetési fáját érintjük **igen** választ adva megállunk. Ha a fa „elfogy” illetve valahonnan megállapítható, hogy a további keresés felesleges, akkor **nem** válasszal állunk meg. Egyébként a bejárás végtelen ideig folytatódhat tovább. A bejárásra ismerünk általános sémákat:

- ▶ preorder bejárás (mélységi/visszalépéses keresés): lehet, hogy nem is terminál, ha belefutunk egy végtelen ágba,
- ▶ szélességi bejárás: mindig terminál, de nem mindig hatékony (ha minden belső csúcsnak legalább 2 gyereke van, akkor $|u|$ függvényében exponenciális ideig tart a keresés).

Egy speciális grammatikaosztályra azonban lehet gyorsan keresni, az LL(k) grammatikákra.

Lefutási fa

Egy grammatika **lefutási fájának** csúcsai az egyes mondatformákhoz tartozó levezetési fák, az élek pedig megfelelnek egyetlen baloldali levezetési lépésnek.



LL(k) grammatikák

Egy grammatika $LL(k)$ grammatika, ha baloldali, kezdőszület egyeztetéses felülről-lefelé elemzést végezve az u még nem egyeztetett részének első k jele alapján egy híján minden alternatíva kizárható. (Az elnevezésben az első L az input balról jobbra olvasására, a második L a baloldali levezetésre utal, míg a k jelentése világos.) Formálisan ez a következőképp fogalmazható meg:

Definició

Egy $G = (N, \Sigma, P, S)$ 2-típusú grammatikát **LL(k) grammatika-nak** nevezünk, ha minden olyan baloldali levezetésre, amelyre

$$S \Rightarrow_G^* uAz_1 \Rightarrow_G uv_1z_1 \Rightarrow_G^* uw_1,$$

$$S \Rightarrow_G^* uAz_2 \Rightarrow_G uv_2z_2 \Rightarrow_G^* uw_2 \text{ és } \text{pre}(w_1, k) = \text{pre}(w_2, k)$$

$(u, w_1, w_2 \in \Sigma^*, v_1, v_2, z_1, z_2 \in (N \cup \Sigma)^*)$ teljesülnek fennáll $v_1 = v_2$.

($\text{pre}(\alpha, k)$ az α szó k hosszúságú prefixét jelöli, ha van legalább k betűje, egyébként pedig α .)

LL(k) grammatikák – Példák

1. Az olyan Greibach normálformájú grammatikák, ahol minden nemterminálisra a szabályalternatívák különböző terminállal kezdődnek LL(1) grammatika.
2. Szintén LL(1) az alábbi grammatika:
 $S \rightarrow aBBBc \mid bC$,
 $B \rightarrow bB \mid aC$,
 $C \rightarrow SS \mid c$.
Egyedül C kérdéses, de SS -ből nem lehet c -vel kezdődő szó.
3. Az alábbi grammatika LL(2):
 $S \rightarrow aBBc \mid bC$,
 $B \rightarrow CB \mid aC$,
 $C \rightarrow abS \mid c$.
Egyedül B kérdéses, az egyetlen lehetséges egyezés az 1. pozícióban, ha CB -ből $abSB$ illetve aC lesz. Azonban a 2. betű már nem lehet egyforma, hiszen C -ből nem lehet b -vel kezdődőt levezetni.

LL(k) grammatikák

Egy LL(k) grammatika esetében elkészíthetők azok a táblázatok, melyek az elemzés pillanatnyi állapota, a (leg)baloldali(bb) nemterminális és az előre olvasott k darab szimbólum alapján egyértelműen meghatározzák a használható alternatívát.

Mindezek következtében az LL(k) grammatikák egyértelmű grammatikák.

Állítás:

LL(k) grammatikák esetében a baloldali, kezdőszelet egyeztetéses felülről-lefelé elemző valamely $u \in L$ szó levezetése során legfeljebb a nemterminálisok számánál eggyel kevesebb lépést tehet úgy, hogy ne hozzon be újabb terminális jelet a mondatforma elejére.

LL(k) grammatikák

Bizonyítás:

Világos, hogy minden baloldali levezetési lépés során egyértelmű, mely szabályt lehet egyáltalán alkalmazni. Tegyük fel, hogy lenne u levezetése során olyan szituáció, amikor a nemterminálisok számával egyező lépést elvégezve az egyeztetett rész hossza nem nőne meg.

Ekkor ezen lépések során - a skatulya elv miatt - volna két olyan mondatforma, amelyben az egyeztetett rész és a legbaloldaliabb nemterminális megegyezik.

Ekkor nyilván az input még nem egyeztetett részének első k jele is ugyanaz, így az LL(k) tulajdonság miatt mindkét helyzetben csak ugyanazzal a szabállyal lehetne folytatni a levezetést. Ettől kezdve a levezetés további része periódikusan ismétlődő levezetés-szekvenciákból állna, s emiatt nem juthatnánk el u -hoz.

□

LL(k) grammatikák

Az előző állításból következik, hogy tetszőleges $u \in L$ szó baloldali levezetéséhez legfeljebb a nemterminálisok számaszor a szó hossza levezetési lépés kell. Emiatt ha ennyi lépésen belül nem vezettük le u -t, akkor az már nem is lesz levezethető, tehát negatív választ lehet adni.

Az előbbieket összefoglalva elmondhatjuk, hogy a baloldali, kezdőszelet egyeztetéses elemző LL(k) grammatikák esetében lineáris idejű elemzést tesz lehetővé.

A gyakorlatban az LL(1) grammatikákat használják, melyek esetében könnyen előállíthatók azok a táblázatok, melyek valamely nemterminális (mint baloldali nemterminális) és egy előre olvasott jel esetén megadják az alkalmazható alternatívát.

Fejlődő rendszerek

Aristid Lindenmayer (1925-1989), magyar származású holland biológus, matematikus a vörös algák (*Callithamnion roseum*) fejlődésének tanulmányozása során írta le a később róla elnevezett párhuzamos fejlődő rendszert.

Az algák sejtjeinek 9 állapota van. Csak ez határozza meg a következő állapotát. Bizonyos állapotokban lévő sejtek növelhetik az algát, sőt oldalágat is növeszthetnek.

Matematikailag így írható le, kezdetben egyetlen 1-es állapotú sejt van.

1 → 23	4 → 25	7 → 8	[→ [
2 → 2	5 → 65	8 → 9[3]	] →]
3 → 24	6 → 7	9 → 9	

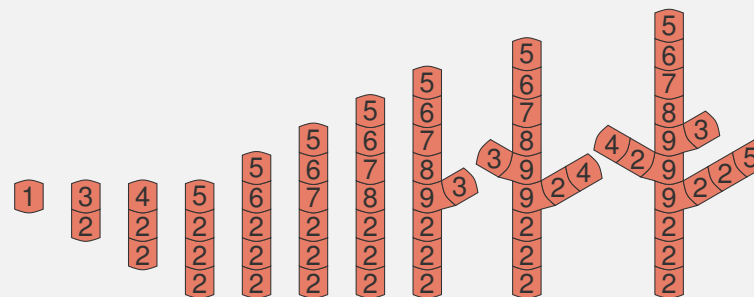
([és] oldalágat határol.)

Vörös algák

1 → 23, 2 → 2, 3 → 24, 4 → 25, 5 → 65, 6 → 7, 7 → 8, 8 → 9[3], 9 → 9, [→ [,] →]

Az 1 axiómából induló levezetés elemei:

1, 23, 224, 2225, 22265, 222765, 2228765, 2229[3]8765
 2229[24]9[3]8765, 2229[225]9[24]9[3]8765
 2229[2265]9[225]9[24]9[3]8765,
 2229[22765]9[2265]9[225]9[24]9[3]8765,
 2229[228765]9[22765]9[2265]9[225]9[24]9[3]8765
 2229[229[3]8765]9[228765]9[22765]9[2265]9[225]9[24]9[3]8765
 2229[229[24]9[3]8765]9[229[3]8765]9[228765]9[22765]9[2265]9[225]9[24]9[3]8765



Determinisztikus Lindenmayer-rendszerek

Definíció

Determinisztikus Lindenmayer-rendszer alatt egy $G = (V, h, \omega)$ rendezett hármast értünk, ahol

- ▶ V egy véges ábécé,
- ▶ $h : V^* \rightarrow V^*$ homomorfizmus
- ▶ $\omega \in V^+$ pedig az axióma (kezdeti szó).

Megjegyzés: Elég h -t V -n megadni.

Definíció

A $G = (V, h, \omega)$ determinisztikus Lindenmayer-rendszer által **generált nyelv** $L(G) = \{h^i(\omega) \mid i \geq 1\}$.

Példa: Legyen $G = (V, h, \omega)$ egy Lindenmayer-rendszer, ahol $V = \{a\}$, $h(a) = a^2$, és $\omega = a^3$.

Ekkor $L(G) = \{a^{3 \cdot 2^i} \mid i \geq 0\}$.

Lindenmayer-rendszer – Példák

Példa: $G_{\text{FIB}} = (\{a, b\}, \{a \rightarrow b, b \rightarrow ab\}, a)$.

Jelölje ω_n az n . átírás után kapott szót.

$\omega_0 = a, \omega_1 = b, \omega_2 = ab = \omega_0\omega_1$. Teljes indukcióval belátjuk, hogy $n \geq 2$ -re $\omega_n = \omega_{n-2}\omega_{n-1}$:

$\omega_{n+1} = h(\omega_n) = h(\omega_{n-2}\omega_{n-1}) = h(\omega_{n-2})h(\omega_{n-1}) = \omega_{n-1}\omega_n$. Tehát

a hosszak $f(n)$ sorozatára $f(0) = 1, f(1) = 1$,
 $f(n) = f(n-1) + f(n-2)$ rekurzió teljesül, ami megegyezik a Fibonacci sorozatra vonatkozó rekurzióval, eggyel eltolva. Tehát $f(n) = F_{n+1}$, ahol F_n jelöli az n . Fibonacci számot.

Teljes indukcióval hasonlóan látható, hogy $N_b(\omega_n) = F_n$ ($n \geq 0$) és $N_a(\omega_n) = F_{n+1}$ ($n \geq 1$).

Példa: Nem létezik olyan G Lindenmayer-rendszer, amelyre $L(G) = \{a, a^2\}$. Ugyanis, ha $\omega = a$, akkor $a \Rightarrow^* a^2$, és így $a^2 \Rightarrow^* a^4$, tehát $a^4 \in L(G)$. Míg ha $\omega = a^2$, akkor $a^2 \Rightarrow^* a$, de ekkor $a \Rightarrow^* \varepsilon$, mert az átírási szabályok környezetfüggetlenek (az egyik a -ból a -t a másikkól ε -t vezettük le), tehát $\varepsilon \in L(G)$.

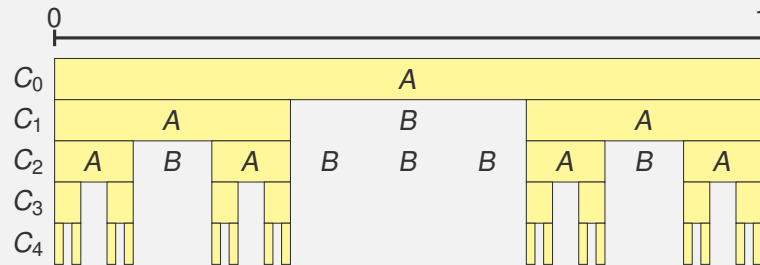
Lindenmayer-rendszer –Példák

Példa: $G_{\text{CANTOR}} = (\{A, B\}, \{A \rightarrow ABA, B \rightarrow BBB\}, A)$.

Legyen $\langle \omega_n, n \geq 0 \rangle$ a G_{CANTOR} rendszer által generált szó sorozat és $\omega_{n,i}$ ($1 \leq i \leq 3^n$) az n . szó i . betűje. Ekkor

$$C_n := \bigcup_{i: \omega_{n,i}=A} \left[\frac{(i-1)}{3^n}, \frac{i}{3^n} \right]$$

a jól ismert Cantor-halmaz közelítésének n -edik iterációja.



Lindenmayer-rendszer –Példák

$G_{\text{SIERPIŃSKI}} = (\{A, B, +, -\}, \{A \rightarrow B - A - B, B \rightarrow A + B + A, + \rightarrow +, - \rightarrow -\}, A)$.

A és B : teknőcgrafika rajzoljon ki egy egységnyi vonalat abba az irányba, amerre a teknős áll. $+$ és $-$ jelentése: forduljon balra illetve jobbra 60 fokkal. Az első páros iterációk ezeket adják:

